This document outlines the Git commands and their functionalities as presented in the video "Git & GitHub Crash Course for Beginners ."

1. Initial Setup and Configuration

- **git --version**: Displays the current version of Git installed on your machine 1.
- **git config --global user.email "[email]"**: Sets your email address globally for all Git projects on your computer. This is required for tracking who makes changes 2.
- **git config --global user.name "[name]"**: Sets your name globally for all Git projects 2.
- **git config --local**: Used instead of --global to apply configuration settings only to a specific repository 3.

2. Starting and Pulling Projects

- **git init**: Initializes a new, empty local Git repository in the current folder, allowing Git to begin tracking changes 4.
- **git clone [url]**: Downloads a copy of a remote repository (from a platform like GitHub) onto your local machine 5, 6.

3. The Staging Area (git add variations)

The process of moving changes from your working directory to the staging area is known as "adding" 7.

- **git add [file_name]**: Stages a specific, individual file 8.
- **git add .**: Stages all changes within the current directory and its subdirectories 8, 9.
- **git add --all** or **git add -A**: Stages every single change across the entire project 10, 11.
- **git add ***: Stages all new or modified files, but **excludes deleted files** 12.
- **git add *.extension**: Stages all files of a specific type (e.g., git add *.txt) in the root folder, excluding deleted files or files in subfolders 8.

4. Saving and Tracking Changes

- **git status**: Provides a summary of which files are modified, new (untracked), or ready to be committed (staged) 7, 10, 13.
- **git commit -m "[message]"**: Permanently saves staged changes to the local repository with a descriptive message of the update 3, 14.
- **git log**: Displays the full history of commits, including IDs, messages, and authors 15.
- **git log --oneline**: Provides a compact, one-line summary of the commit history 15.
- **git diff [commit_id1] [commit_id2]**: Compares the specific line-by-line differences between two commits 16.

5. Undoing and Removing

- **git reset**: Moves files from the staging area back to the working directory (unstaging) 9, 11.

- **git reset HEAD~**: Undoes the last commit and returns those changes to the working directory 17.
- **git reset --hard**: A forceful command that restores both staged changes and deleted files to their last committed state 18, 19.
- **git rm [file_name]**: Deletes a file and stages that deletion in one step 20.
- **git rm -f [file_name]**: Forcefully deletes a file that has uncommitted local modifications 19.
- **git rm --cached [file_name]**: Removes a file from Git's tracking/staging area but **keeps the actual file** on your computer 19.
- **git rm -r [folder_name]**: Recursively removes a folder and all its contents 21.
- **git restore [file_name]**: Reverts an uncommitted file to its state as of the last commit 22.
- **git restore .**: Reverts every file in the project to the last committed version 23.
- **git restore --staged [file_name]**: Removes a file from the staging area while keeping the changes in the working directory 23.
- **git revert [commit_id]**: Undoes a specific commit by creating a **new commit** that reverses the changes, keeping the project history intact 24, 25.

6. Branching and Merging

- **git branch**: Lists all branches in the repository; the current branch is marked with an asterisk 26.
- **git branch [branch_name]**: Creates a new branch from the current state 26.
- **git checkout [branch_name]**: Switches the working environment to the specified branch 26, 27.
- **git checkout [commit_id]**: Switches the project to the exact state of a specific previous commit (Head Detached state) 28.
- **git merge [branch_name]**: Combines changes from the specified branch into the branch you are currently on 29, 30.
- **git rebase [branch_name]**: Rewrites the history of the current branch by applying its commits on top of the latest commits from the specified branch, creating a clean linear history 31-33.

7. Remote Operations (GitHub)

- **git push origin [branch_name]**: Sends your local commits to the specified branch on the remote server (GitHub) 34.
- **git fetch**: Downloads updates from the remote repository to local memory without merging them into your files 35, 36.
- **git pull**: A combination of fetch and merge; it downloads remote changes and immediately updates your local files 35, 37.

8. Stashing

Used to temporarily set aside unfinished work to switch branches without committing 38, 39.

- **git stash**: Saves uncommitted changes into a temporary area (the stash) and clears the working directory 39.
- **git stash list**: Shows all saved stashes with their unique identifiers 40, 41.

- **git stash pop**: Restores the most recent stash and **removes it** from the stash list 39, 42.
- **git stash apply**: Restores the stash but **keeps it** in the list for future use 24, 40.
- **git stash drop**: Deletes a specific stash from the list 41.

Note on Terminal Commands: The sources also mention general terminal commands used during the tutorial that are *not* Git-specific: cd (change directory), mkdir (make directory), ls (list files), ls -la (list all files including hidden), pwd (print working directory), touch (create file), and open . (open folder in finder) 6, 43-45.